

# Assignment 5: A Trip to Monte Carlo

---

Using the [Monte Carlo method](#) for complex estimations!

1.  **Worth:** 0% (OPTIONAL). Completing this assignment can boost the grade of your worst Assignment (A1/A2/A3/A4) grade.
2.  **Due Date:** See LEA
3.  **Late submissions:** N/A
4.  **Submission:**
  - Put all .py file(s) into a .zip and submit on LEA
  - Please use the provided files **WITHOUT** renaming them.
  - Fill the docstring header at the top of each .py file with your name, section and date.
  - Use the `my_functions.py` to define all your functions
  - Use the `assignment5.py` file to test all your functions.

In this assignment, we will be using a [Monte Carlo Simulation](#) to estimate the area of complex polygons.

There are sixteen different polygons that are included in this assignment, but you will only need to concern yourself with two specific polygons. Your job is to estimate the area of these polygons.

```
import image_lists_final

# =====
# modify your student id appropriately
# =====
student_id = 1234567

# =====
# get the images
# =====
my_image = image_lists_final.images[student_id % 16]
test_image = image_lists_final.test_image
```

Each polygon is a list that contains points, where each point has an  $(x,y)$  value.

```
for x,y in test_image:
    print(f"Point is ({x},{y})")

# individual point
x,y = test_image[3]
```

## Submission

---

Please submit TWO files

- `verification.py` - this file will be provided to you, but you may need to make small changes
- `my_functions.py` - this file (exact name!) needs to be created by you, and it must contain all the functions that are described in the instructions below.
  - You are permitted to create additional functions

## Definition: Monte Carlo Simulation

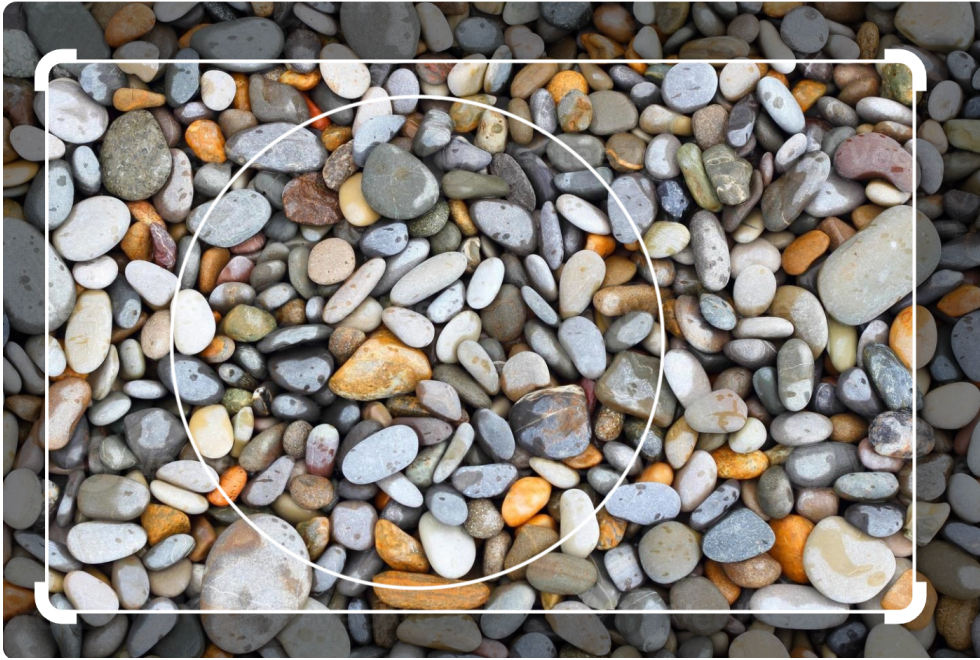
---

A *Monte Carlo Simulation* can be a computer program that uses randomized inputs to determine what a particular outcome might be in real life. The simulation is named after the famous city “Monte Carlo” which is one of the most famous gambling cities in the world.

## Example: Estimate the area of a circle

---

Imagine we have a circle inscribed on a rocky beach. We want to know the area of this circle, approximately.



If the rocks are small and evenly distributed, then the ratio of visible rocks inside the circle to the visible rocks in the rectangle would be almost equal to the ratio of area of the circle to the area of the rectangle.

$$\frac{N_C}{N_R} \approx \frac{A_C}{A_R}$$

where:

- $N_C$  is the number of rocks within the circle,
- $N_R$  is the number of rocks within the rectangle
- $A_C$  is the area of the circle
- $A_R$  is the area of the rectangle

$$A_C \approx A_R \cdot \frac{N_C}{N_R}$$

If we have *enough* rocks, we should be able to estimate the value of  $\pi$

## Part 1 - Does a horizontal arrow intersect a given line segment?

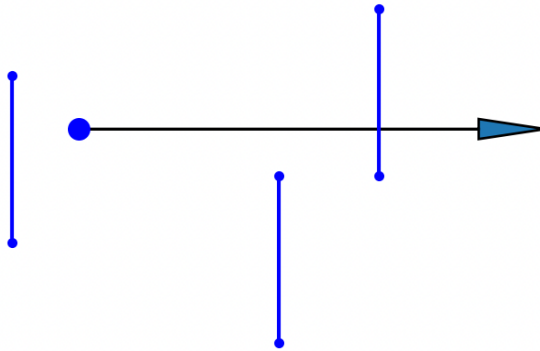
---

Assume the arrow starts at a given point (  $x$  ,  $y$  ) and continues to the right to infinity

### Hints

Break the problem into 3 separate conditions

- line segment is vertical, which lines are crossed by the arrow? Why?



- line segment is horizontal
  - in this assignment, just ignore
- if line segment is neither vertical or horizontal

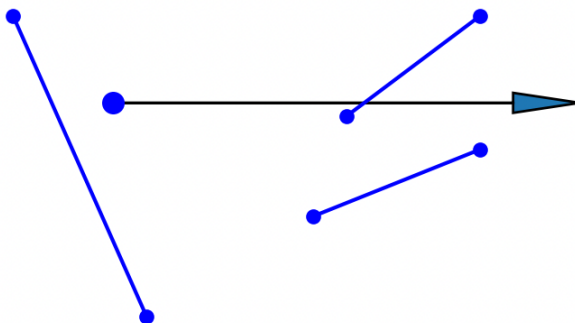
Line Equations:

$$y = mx + b$$

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$b = y_1 - mx_1$$

If the lines were infinite, where would they intersect? From this, can we determine if the arrow crosses the line segment?



## Instructions

Write a function `cross_line` which takes in the following arguments:

- `x` the x position of the starting point of the arrow
- `y` the y position of the starting point of the arrow
- `x1` the x position of the point 1 of the line segment

- `y1` the y position of the point 1 of the line segment
- `x2` the x position of the point 2 of the line segment
- `y2` the y position of the point 2 of the line segment

and returns:

`True` if the arrow intersects the line, `False` otherwise.

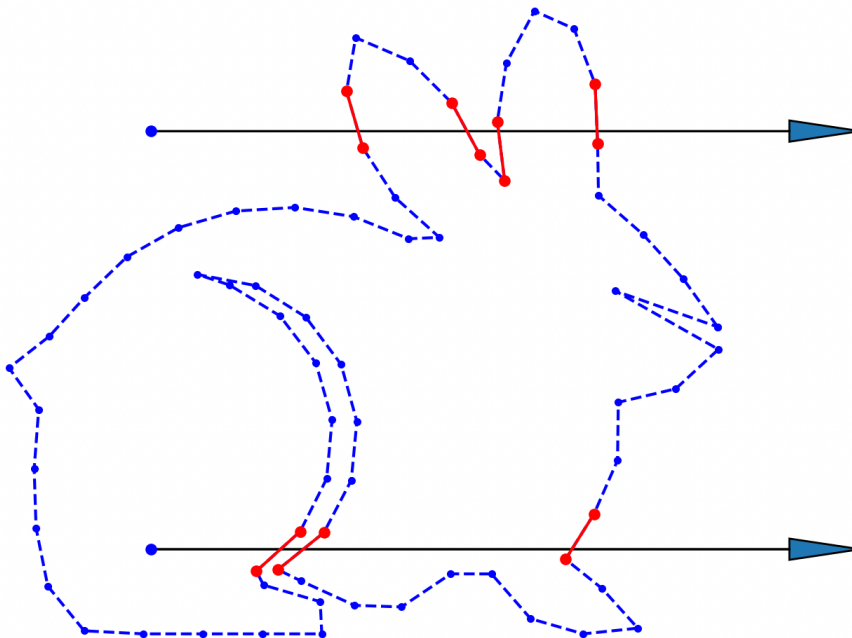
## Part 2 - Is a Point Within a Polygon?

---

### Theory

Pick a point. Is it in the polygon?

- Draw an arrow horizontally starting from the given point, and go in one direction to infinity.
- Does the this *arrow* intersect an even number of lines along its path?
  - The original point is not within the polygon
- Does the this *arrow* intersect an odd number of lines along its path?
  - The original point is within the polygon



### Instructions

Write a function `is_in_polygon` which takes in the following arguments:

- `x` the x position of the starting point of the arrow

- `y` the y position of the starting point of the arrow
- `points` a list of points `[(x1,y1), (x2,y2), ...]` describing the polygon

and returns:

`True` if the initial point is within the polygon, `False` otherwise.

## Part 3 - Estimate Area of a Polygon using Monte Carlo Methods

---

Using a Monte Carlo simulation, randomly throw `rocks` inside of square which encloses a polygon. By counting how many rocks landed inside the polygon versus how many land in the square, estimate the area of the polygon.

NOTE:

To use random numbers:

```
# use the random Library
import random

# get a random number somewhere between start and end, using an even probability
x = random.uniform(start, end)
```

NOTE:

To plot

```
import matplotlib.pyplot as plt

"""Plot the results of the rock throwing"""
_, axes = plt.subplots()

# random rocks
axes.plot(xdata_inside, ydata_inside, 'bo', markersize=1)
axes.plot(xdata_outside, ydata_outside, 'ro', markersize=1)
axes.set_aspect('equal', adjustable='box')
plt.show()
```

## Instructions

Create a function `throw_rocks` that takes in the following arguments

- `n` the number of rocks to throw
- `max_size` the width and height of the square enclosing the polygon
- `points` a list of points `[(x1,y1), (x2,y2), ...]` describing the polygon
- `make_plot` a `True/False` boolean which is used to determine whether or not we should plot the result

and returns

- the estimated area of the polygon

This function should plot all the rocks thrown (if requested). If the rock lands inside of the polygon, it should be coloured blue, otherwise it should be coloured red.

## Verification

To see if what you did is correct... (images shown have 500 and 5000 rocks thrown respectively )

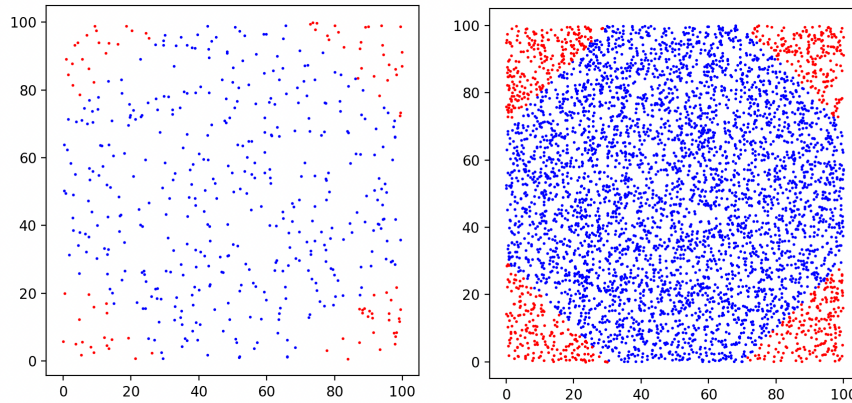
```
import image_lists_final
from my_functions import throw_rocks

test_image = image_lists_final.test_image
number_of_rocks = 500 # you can change this if you want
max_size_of_image = 100 # this is kinda fixed

area = throw_rocks(number_of_rocks, max_size_of_image, test_image, plot=True)
print(area)
```

Note that because of the randomness, you will not get the same area, although it should be approximately this number

8292.0



## Part 4 - How Consistent is the Calculation?

---

### Instructions

Write a function `estimate_100_areas` that takes in the following arguments

- `n` the number of rocks to throw per estimate
- `max_size` the width and height of the square enclosing the polygon
- `points` a list of points `[(x1,y1), (x2,y2), ...]` describing the polygon result

and returns

- a list containing 100 estimates of the area of the polygon

The function should call `throw_rocks` 100x and save the results in a list, which is then subsequently returned

### Verification

To see how good this is at estimating the area, we are going to use the `statistics` library, and determine the `mean` (average) and `stdev` (standard deviation).

```
import statistics
import image_lists_final
from my_functions import throw_rocks, estimate_100_areas

test_image = image_lists_final.test_image

number_of_rocks = 500 # you can change this if you want
max_size_of_image = 100 # this is kinda fixed

# what is the mean value of the area and its standard deviation if your
# estimate the area 100 times in a row?
```



```
areas = estimate_100_areas(number_of_rocks, max_size_of_image, test_image)
standard_dev = statistics.stdev(areas)
average = statistics.mean(areas)

# print the estimate area (average) and the PERCENTAGE of the standard deviation
print(f"The area is approximate {average:.2f} ± {standard_dev / average * 100:.2f} %")

# Note, for the test image (you may get a slightly different result):
# The area is approximate 8201.80 ± 1.5 %
```

## Part 5 - Summary

---

In the `verification.py` file, at the end, in the comment section, answer the following two questions.

1. What image does your polygon draw? In other words, what is it?
2. If you really hated calculus, how could you use this method to get a definite integral of a given function?